BoxLang + AI Hackathon: los 10 challenges

Resumen en español con nivel de dificultad · ANIA x Ortus Solutions · 10 y 11 de octubre de 2026, El Salvador

Cómo se calculó la dificultad. Escala de 1 a 5 pensada para un equipo de estudiantes en un día de hackathon, usando lo que ya verificamos con BoxLang 1.18 + bx-ai 3.6 (chat, tools, agentes con memoria, salida estructurada, RAG simple con aiDocuments, servidor web y proveedor mock). Pesa más lo que no está probado en nuestro kit (BX Agents, búsqueda web, bases de datos, embeddings) y el riesgo de que la demo falle en vivo.

#	Challenge	En una frase	Dificultad
01	**Incident Zero**	El sistema en producción falla y nadie sabe por qué.	**Media** (3/5)
02	**Agent Factory**	Un agente que construye agentes.	**Alta** (5/5)
03	**Ship It**	Convertir una idea en software funcional.	**Media-alta** (4/5)
04	**Rabbit Hole**	Investigar algo difícil y demostrar las conclusiones.	**Media** (3/5)
05	**Break Me**	Tu trabajo es hacer fallar a otra IA.	**Media** (3/5)
06	**Data Miner**	Aquí están los datos. Dinos qué importa.	**Media** (3/5)
07	**Second Brain**	Convertir un montón de información en conocimiento usable.	**Media-baja** (2/5)
08	**HELP!**	Al cliente no le importa qué departamento lo atiende. Solo quiere que se resuelva.	**Media-baja** (2/5)
09	**Agent Toolbox**	Los agentes valen lo que valen sus herramientas. Dales algo nuevo.	**Media** (3/5)
10	**This Sucks**	Encuentra algo que los desarrolladores odian hacer y mejóralo.	**Variable** (3/5)

Recomendación para ganar. Las mejores apuestas por relación impacto/riesgo son **08 HELP!** (casi todo ya está en el starter web), **07 Second Brain** (RAG y loaders probados) y **05 Break Me** (demo muy vistosa y tema de seguridad que el jurado valora). **06 Data Miner** es buena si alguien del equipo domina SQL. Evitaría **02 Agent Factory** salvo que el equipo ya conozca BX Agents.

01 · Incident Zero

Media (3/5)

El sistema en producción falla y nadie sabe por qué.

Qué piden: Un sistema de respuesta a incidentes con IA que investigue una falla desconocida revisando logs, código, configuración y documentación.
Mínimo: Identificar la causa raíz probable con evidencia. **Extra:** Agentes especializados (logs, infraestructura, código, diagnóstico).
Piezas de BoxLang AI sugeridas: Agentes, tools, subagentes, RAG, salida estructurada, pipelines, HITL.
Por qué esta dificultad: Las piezas existen y las probamos: tools que leen archivos y buscan en logs, y salida estructurada para el informe (causa, evidencia, confianza). Lo difícil es preparar una app de ejemplo con una falla creíble y lograr que el agente no invente evidencia.
Enfoque viable en un día: Una app rota de juguete + 3 tools (leerLog, buscarEnCodigo, leerConfig) + un agente que devuelve un reporte estructurado citando líneas.

02 · Agent Factory

Alta (5/5)

Un agente que construye agentes.

Qué piden: Recibe la descripción de un problema y genera un proyecto BX Agents funcional (tools, instrucciones, skills, modelo, subagentes, estructura).
Mínimo: Generar un proyecto BX Agents ejecutable a partir de lenguaje natural. **Extra:** Ejecutar y evaluar automáticamente el agente generado.
Piezas de BoxLang AI sugeridas: Agentes, salida estructurada, tools, generación de código, subagentes, evaluación.
Por qué esta dificultad: Exige que el código generado corra a la primera, y BX Agents no está en nuestro kit verificado (solo bx-ai). Hay que aprender su estructura de proyecto el mismo día. Mucho riesgo de que la demo falle en vivo.
Enfoque viable en un día: Plantillas fijas de proyecto + el LLM solo rellena huecos (instrucciones, lista de tools) con salida estructurada. Así se reduce lo que puede fallar.

03 · Ship It

Media-alta (4/5)

Convertir una idea en software funcional.

Qué piden: Un equipo de desarrollo con IA que lleve una petición de feature a código: roles de requisitos, arquitectura, implementación, pruebas, documentación y revisión, decidiendo dónde interviene el humano.
Mínimo: Aceptar una feature y producir plan de implementación y código funcional. **Extra:** Probar, revisar, documentar o crear un cambio listo para pull request.
Piezas de BoxLang AI sugeridas: Multi-agente, tools, skills, salida estructurada, pipelines, HITL.
Por qué esta dificultad: La parte multi-agente es fácil de armar con subAgents, pero que el código generado funcione y pase pruebas es frágil. El alcance es enorme; hay que recortarlo a un tipo de feature concreto.
Enfoque viable en un día: Pipeline planificador > programador > revisor sobre una mini-app BoxLang, con aprobación humana entre pasos y ejecución de las pruebas.

04 · Rabbit Hole

Media (3/5)

Investigar algo difícil y demostrar las conclusiones.

Qué piden: Un sistema que planifique la investigación, junte evidencia de varias fuentes, evalúe las fuentes, detecte información contradictoria y llegue a una conclusión respaldada.
Mínimo: Un reporte de investigación con evidencia rastreable. **Extra:** Investigadores independientes y otro agente que evalúe sus conclusiones.
Piezas de BoxLang AI sugeridas: Agentes, tools, pipelines, salida estructurada, subagentes, memoria.
Por qué esta dificultad: Existe aiWebSearch, pero necesitas un proveedor de búsqueda con key y no lo probamos. Las citas exactas y la detección de contradicciones requieren buen diseño de prompts. Como plan B se puede investigar sobre un set de documentos local.
Enfoque viable en un día: Planificador que divide la pregunta, 2 investigadores en paralelo (aiParallel / subAgents) y un juez que puntúa fuentes y marca contradicciones.

05 · Break Me

Media (3/5)

Tu trabajo es hacer fallar a otra IA.

Qué piden: Un sistema de red team que pruebe otro agente: prompt injection, abuso de tools, filtración de información, documentos maliciosos, manipulación de instrucciones, consumo excesivo o acciones no autorizadas.
Mínimo: Probar un agente automáticamente y generar un reporte de seguridad. **Extra:** Atacante y defensor que se adaptan entre sí.
Piezas de BoxLang AI sugeridas: Agentes, tools, middleware, salida estructurada, HITL, evaluación.
Por qué esta dificultad: Muy demostrable: se ve en vivo cómo el agente cae o resiste. Se puede armar la víctima con el starter (agente con tools y un secreto) y el atacante como otro agente. Funciona con el proveedor mock para desarrollar. Lo exigente es que el reporte sea serio (casos, severidad, evidencia).
Enfoque viable en un día: Agente víctima con una tool 'peligrosa' + corpus de ~20 ataques + atacante que muta los que fallan + reporte estructurado; luego mostrar la defensa con aiFence.

06 · Data Miner

Media (3/5)

Aquí están los datos. Dinos qué importa.

Qué piden: Un analista autónomo que inspeccione un dataset o base de datos desconocida, formule preguntas, ejecute consultas, analice resultados y separe hechos de interpretación.
Mínimo: Responder preguntas en lenguaje natural y descubrir al menos un insight útil por su cuenta. **Extra:** Agentes especializados de SQL, estadística, visualización o verificación.
Piezas de BoxLang AI sugeridas: Agentes, tools, salida estructurada, pipelines, memoria.
Por qué esta dificultad: Patrón clásico: tool 'ejecutarSQL' de solo lectura + tool 'verEsquema'. Hay que conectar una base (SQLite/H2 vía datasource, no probado en el kit) o empezar con CSV. El reto real es que el agente no invente números: cada cifra debe salir de una consulta.
Enfoque viable en un día: Dataset público local (ej. datos abiertos de El Salvador), tools de esquema y SELECT, respuestas con la consulta usada y etiqueta 'hecho' vs 'interpretación'.

07 · Second Brain

Media-baja (2/5)

Convertir un montón de información en conocimiento usable.

Qué piden: Ingerir documentos, código, notas, Markdown, PDF, JSON, etc. Un agente que conecte información entre fuentes, responda preguntas, detecte contradicciones y separe hechos de inferencias con evidencia.
Mínimo: Ingerir varios tipos de contenido y responder preguntas con evidencia. **Extra:** Ingesta continua o relaciones entre conceptos.
Piezas de BoxLang AI sugeridas: RAG, embeddings, vector stores, document loaders, agentes, memoria.
Por qué esta dificultad: Es lo que más cubre bx-ai de serie: aiDocuments carga .md, .txt, .pdf, .csv y .json, y el RAG simple ya está probado en el starter (04-rag.bxs). Los vectores necesitan un proveedor con embeddings. Para destacar hay que ir más allá: citas por fragmento y detección de contradicciones.
Enfoque viable en un día: Carpeta con apuntes/reglamentos mezclados + RAG con chunks + respuesta con citas (archivo y fragmento) + modo 'buscar contradicciones'.

08 · HELP!

Media-baja (2/5)

Al cliente no le importa qué departamento lo atiende. Solo quiere que se resuelva.

Qué piden: Un sistema de soporte con IA que se haga cargo del problema del cliente de principio a fin, usando documentación, datos de cuenta, pedidos, diagnósticos o agentes especialistas.
Mínimo: Clasificar la solicitud, juntar la información necesaria y recomendar una solución. **Extra:** Elegir especialistas dinámicamente, ejecutar acciones aprobadas y escalar a un humano cuando haga falta.
Piezas de BoxLang AI sugeridas: Agentes, subagentes, RAG, tools, memoria, HITL.
Por qué esta dificultad: Es el más cercano al starter web que ya funciona (chat + agente con memoria por sesión + tools). Los datos de clientes/pedidos se pueden simular en JSON. Demo fácil de entender para el jurado. Riesgo: que se vea como 'un chatbot más' si no se muestran el ruteo y el escalamiento.
Enfoque viable en un día: Agente triaje que deriva a subagentes (facturación, técnico, pedidos), tools sobre datos falsos, acción 'reembolso' que pide aprobación humana.

09 · Agent Toolbox

Media (3/5)

Los agentes valen lo que valen sus herramientas. Dales algo nuevo.

Qué piden: Una capacidad reutilizable para BoxLang AI o BX Agents que no exista o mejore mucho una existente (integraciones con plataformas, bases de datos, nube, documentos, mensajería, navegador, APIs, tooling).
Mínimo: Una capacidad documentada y probada que otra app de BoxLang AI pueda usar. **Extra:** Hacerla genérica, configurable, segura y publicable.
Piezas de BoxLang AI sugeridas: Cualquiera.
Por qué esta dificultad: El alcance técnico puede ser pequeño (un módulo de tools bien hecho), pero piden tests, documentación y calidad publicable. Hay que investigar qué falta en el ecosistema para no duplicar algo. La demo es menos vistosa: conviene acompañarla de un agente que la use.
Enfoque viable en un día: Un paquete de tools para un servicio concreto (ej. Google Sheets, Telegram o SQL seguro) con README, tests y un ejemplo de agente.

10 · This Sucks

Variable (3/5)

Encuentra algo que los desarrolladores odian hacer y mejóralo.

Qué piden: Identificar una tarea de desarrollo frustrante, repetitiva o lenta y resolverla con BoxLang AI o BX Agents. No hay problema prescrito; la IA debe aportar valor real y el código tradicional se usa donde sea mejor.
Mínimo: Demostrar un problema real y una solución funcional. **Extra:** Hacer la solución reutilizable fuera del equipo.
Piezas de BoxLang AI sugeridas: Lo que resuelva el problema.
Por qué esta dificultad: La dificultad la decides tú. Es buena opción si el equipo ya tiene un dolor concreto (mensajes de commit, documentar código, traducir errores de stack, migrar CFML). El riesgo es elegir un problema que el jurado no sienta como real.
Enfoque viable en un día: Herramienta CLI en BoxLang que explique un stack trace y proponga el arreglo, o que genere tests/README de un archivo.